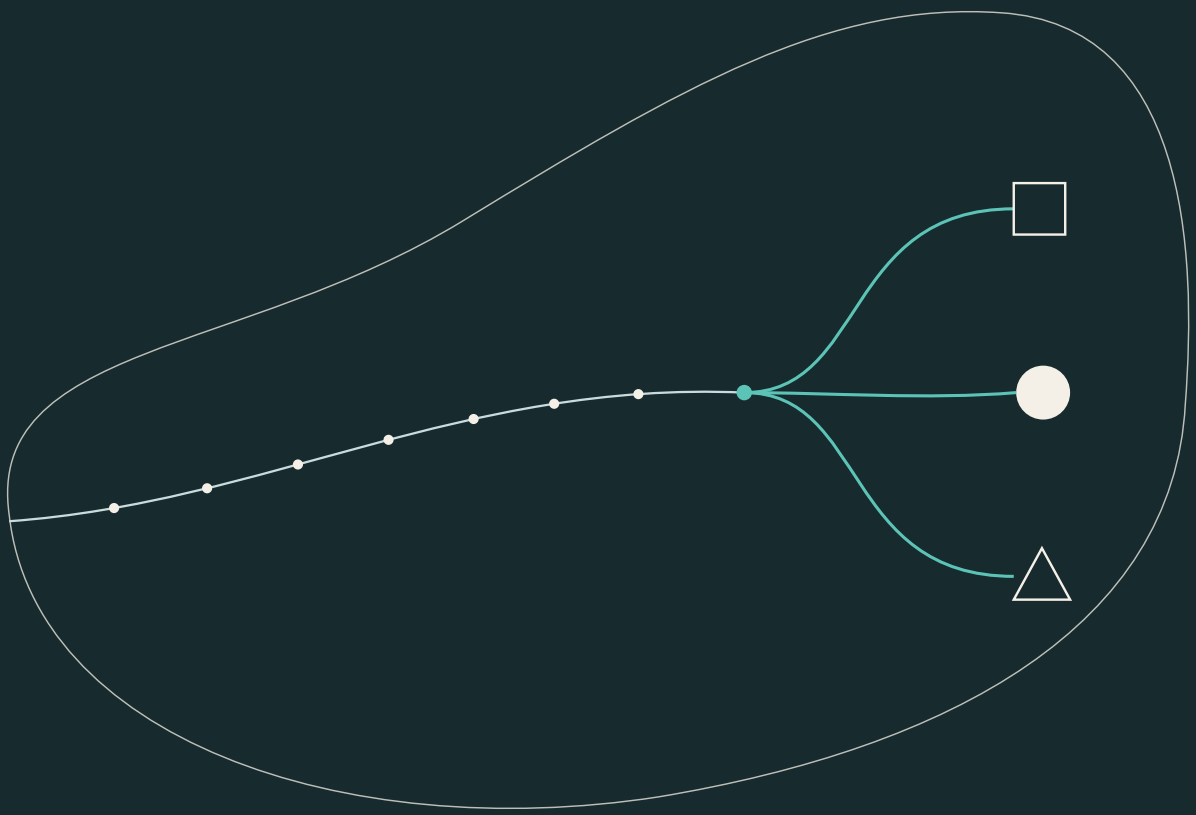
QwenDec

Thinking fast and slow. In one model.



Progress report / November 2026

One mind, two tempos.

QwenDec is building a language model that thinks at two speeds with one set of weights.

Slow. Reasoning, one token after another.

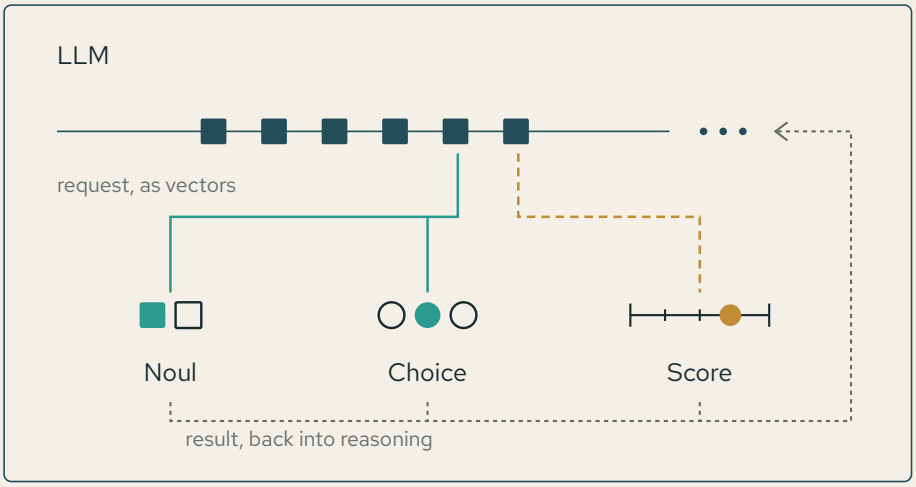
Fast. Noul, Choice and Score: a decision in one step.

A language model reasons slowly. It writes one token, then the next. Some questions inside that reasoning do not need a paragraph: is this true, which of these, how much.

QwenDec gives the model a fast way to settle them. These are JEV-style decisions, answered in one step as a value and made inside the model while it thinks.

It stays one model. The prompt is read once. When a decision is needed, the request and the answer pass inside the model as vectors, never as written text.

ONE MODEL



● Working ○ Pending ◌ To build

The goal, marked by its state today. Noul and Choice decide from requests that arrive as vectors. Score is open. The model does not yet ask for a decision or take the result back by itself.

What works. What comes next.

Working

- **Noul** — 1,162 / 1,182
  Is it true? Asked as vectors, not text.
- **Choice** — 955 / 1,001
  Which one? Asked as vectors, not text.
- **Deciding from what the model already read** — 69 / 81
  Recognises an unresolved question. Before: 2 of 81.
- **Training without the CPU** — 0.04%
  Share of one CPU core during a full training run.

Pending

- **One adapter for both** — 1,128 · 929
  Passed its rule narrowly. Acceptance pending.

Still to build

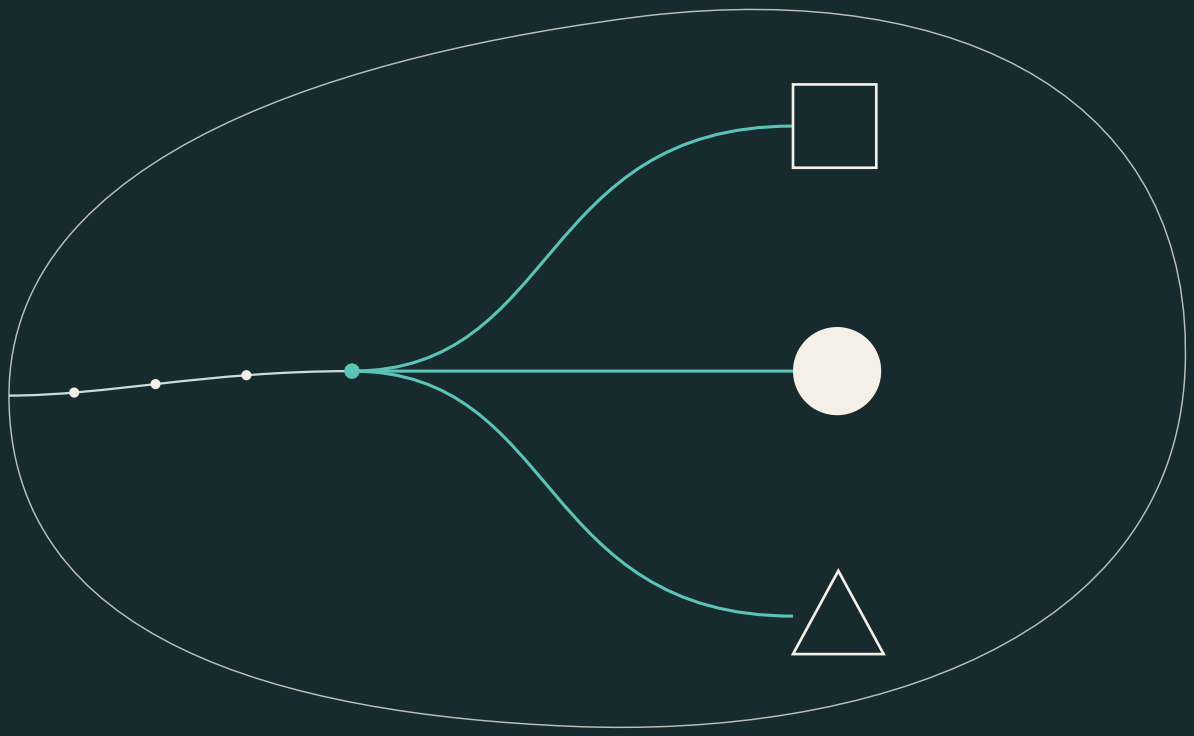
- **Reliable Score**
  In-between values are poor.
- **Model asks + uses answers**
  Today the request is handed to the model.
- **Decision budget prediction**
- **Runtime + accounting**

Fractions are questions answered correctly on development tests. All cases are synthetic. Pages 10 to 12 and 19 say what the figures do and do not show.

CHAPTER I

The fast tempo.

Three decisions, and how they came to read what the model has already read.



Three decisions.

Each answers one bounded question with a value, in one step.



Noul

Is it true?

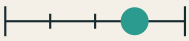
The probability that a statement is true, given the evidence.



Choice

Which one?

One of several alternatives. The same one, whatever order they are listed in.



Score

How much?

A value between 0 and 1, measured against a rubric.

A decision can decline

Only a decision that can be assessed carries a value. Otherwise it says why not.

Insufficient

The evidence does not reach the question.

Conflicting

The evidence points both ways.

Unresolved

The question or rule can be read in more than one way.

No feasible choice

Choice only: none of the alternatives qualifies.

Knowing when not to answer was the largest gap in the adapter QwenDec started from, and the first thing it repaired. The next page shows by how much.

04 THE MODEL'S OWN READING

Deciding from what was already read.

UNRESOLVED QUESTIONS RECOGNISED

69 / 81

Evidence / synthetic cases

Before: 2 of 81. The decision now works from the model's own reading of the prompt.

The published decision adapter expects to read the whole prompt itself. In QwenDec the model reads the prompt natively, as it always does, and nothing is read twice. The decision adapter switches on only for the question and its options.

One training run on 10 November taught the adapter to decide from that state.

BEFORE AND AFTER

Development cases	Before	After	Of
Recognises "unresolved"	2	69	81
Recognises "conflicting"	47	80	85
Noul: status and value right	194	228	243
Choice: status and value right	165	225	232
Score: status and value right	111	133	199

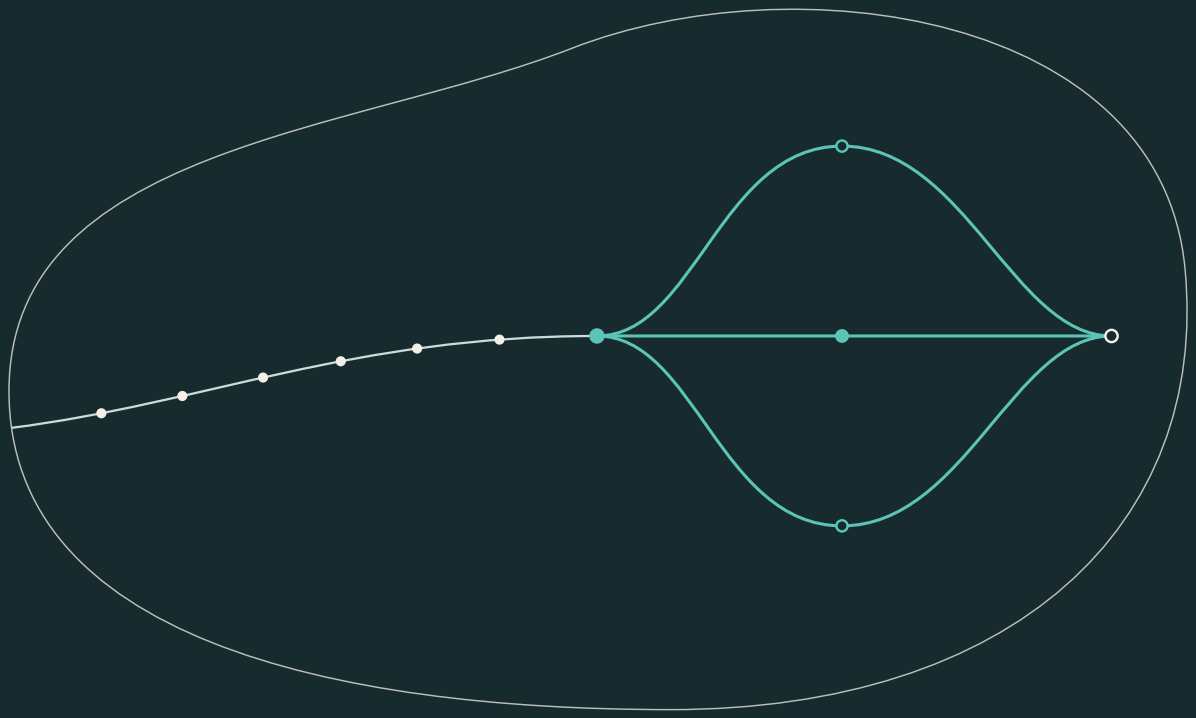
● Working. Accepted on 11 November for Noul, Choice and recognising when not to answer.

○ Score did not follow. In-between values: 11 of 67 right, against 15 before. See page 17.

CHAPTER II

Asking without words.

A request that crosses as vectors, not text. It took four attempts.



How a request crosses.

A request is 16 vectors and a pointer. No sentence is written.

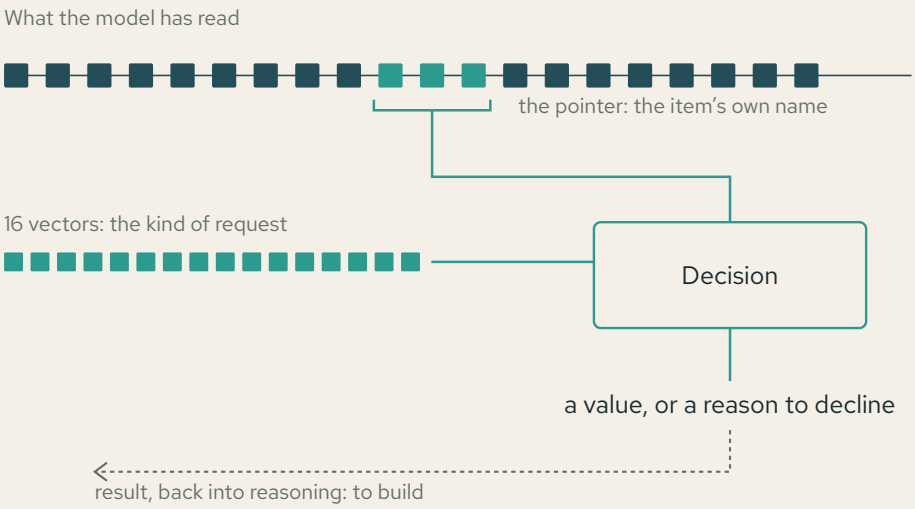
The kind of request. One of a few fixed kinds: six for Noul, two for Choice. Each is 16 vectors.

The item. A pointer to the item's own name in what the model has read.

The request says two things. What kind of question is being asked, and which item it is about.

The kind crosses as vectors. The item is not described again: the request points at the place where the model already read its name.

ONE REQUEST



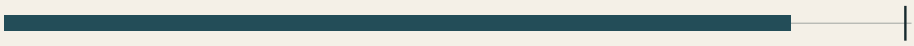
Why a pointer. Vectors alone carried the kind of question but not the item: with two items in play, the answer was right no more often than chance. Pointing at the item's own tokens fixed that.

Four attempts.

The format passed on the fourth attempt. The first three did not, and each failure said what to change.

1

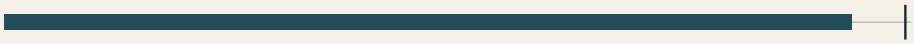
1,024 / 1,182 — Not passed



Vectors alone. Right kind of question, wrong item: 35 of 72 where two items differ, which is chance.

2

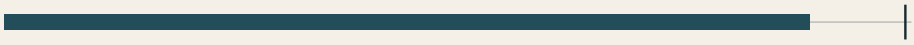
1,104 / 1,182 — Not passed



A pointer added. The item is now found: 72 of 72. It fell short of the 1,115 needed, on questions worded freely.

3

1,049 / 1,182 — Not passed



Six fixed requests. Free wording repaired: 153 of 184. A negation was lost on two names that differ in one word.

4

1,162 / 1,182 — Passed



The same design, run again, judged on distinct names. All six parts of its rule met.

Evidence / synthetic cases

Bars run from 0 to 1,182 questions. The tick marks 1,174, the score when the same questions are read as text. Attempts 1 to 3 were judged on the original item names, attempt 4 on distinct names; the text score is 1,174 on both.

Attempts 3 and 4 are one design. Only the order of the training steps differs. On the same questions they score 1,132 and 1,162: how far two runs can differ is still an open question.

07 RESULT

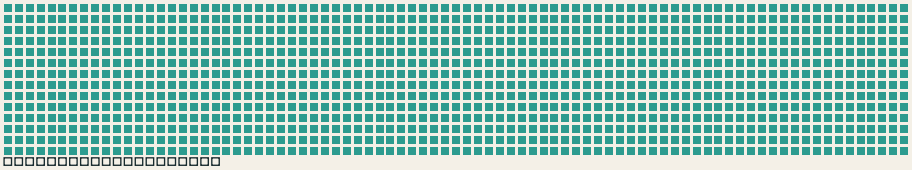
Noul.

QUESTIONS RIGHT

Evidence / synthetic cases

1,162 / 1,182

Against 1,174 when the same questions are read as text.



One square for each question. The 20 hollow ones, in the last row, were answered wrong.

WHAT HOLDS

● A question and its negation, both right	425 of 428	text 427
● The answer follows the pointer when it moves	72 of 72	
● Questions held back from training	600 of 600	
● Two item names that differ in one word	1,160 of 1,182	

WHERE IT IS WEAKER

○ Questions worded freely	165 of 184	text 176

19 of its 20 errors are on freely worded questions, a wording the training cases do not contain.

● Working. The request format was accepted for Noul on 11 November, as the basis for the next stage. Reliability across runs was left open.

08 RESULT

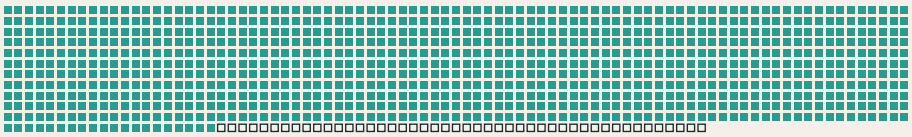
Choice.

QUESTIONS RIGHT

Evidence / synthetic cases

955 / 1,001

Against 971 when the question and the alternatives are read as text.



One square for each question. The 46 hollow ones, in the last row, were answered wrong.

WHAT HOLDS

● The same answer in every order of the alternatives	109 of 112	text 103
● Picks the right alternative when there is one	312 of 319	text 301
● "Which to pick" and "which is ruled out", both right	352 of 374	text 367

WHERE IT IS WEAKER

○ Recognises that no alternative qualifies	90 of 100	text 100

Asked which to pick when none qualifies, it sometimes names one anyway. The text version does not make this mistake.

The request never restates an alternative's properties. It only points at each alternative's name; the properties stay where the model read them. On two measures this does better than text.

- Working. The request format was accepted for Choice on 11 November, on the same terms as Noul.

09 RESULT

One adapter for both.

One adapter now serves both kinds of request. It passed its rule, narrowly.

QUESTIONS RIGHT

Evidence / synthetic cases

Noul	Choice
1,128 / 1,182	929 / 1,001
Needed 1,115. Separate adapter: 1,162.	Needed 921. Separate adapter: 955.

THE RULE, PART BY PART

Part	Needed	Measured	Margin
● Noul: a question and its negation	385	406	+21
○ Noul: all questions	1,115	1,128	+13
○ Noul: free-form questions	151	153	+2
● Noul: the two items differ	65	72	+7
● Noul: answer follows the pointer	65	72	+7
○ Choice: all questions	921	929	+8
● Choice: pick and rule out, both right	331	342	+11
● Choice: right in every order	93	105	+12

All eight parts are met. Three by small margins: 2, 8 and 13 questions, marked with a ring.

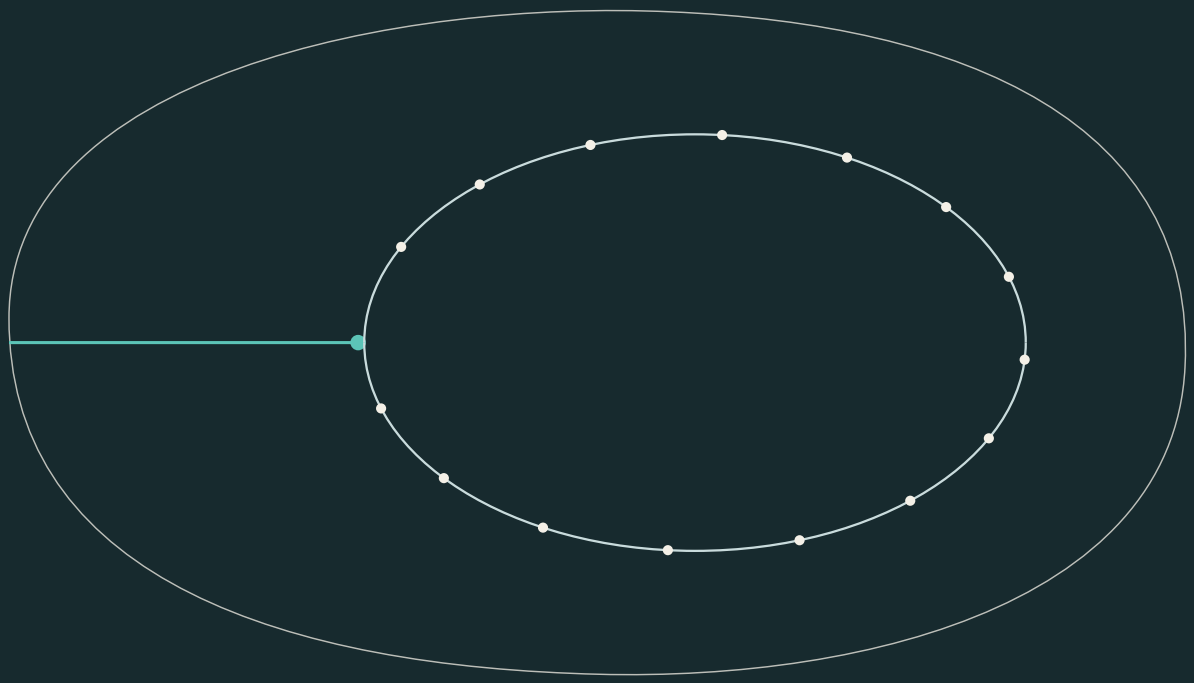
It scores below both separate adapters. Whether that is a cost of training the two together is not shown: two Noul-only runs of one design differ by about as much.

○ Acceptance pending. The decision is on page 18.

CHAPTER III

Training without the CPU.

The training loop runs on the graphics card from the first step to the last.



10 THE TRAINING LOOP

The loop lives on the GPU.

Training no longer waits for the CPU.

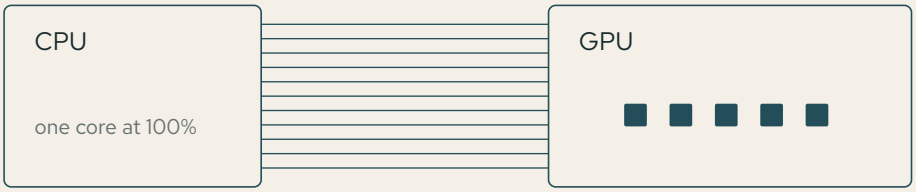
In the first attempt at this project, one CPU core ran at full load throughout training and held the graphics card back. It could not be removed afterwards.

QwenDec therefore started from the other end. First a training loop that does not involve the CPU, then everything else.

In ordinary training the CPU sends the GPU every small operation, one at a time. Here one whole training step is recorded once: pick the batch, compute, correct the weights, write the log. The GPU then repeats that step by itself.

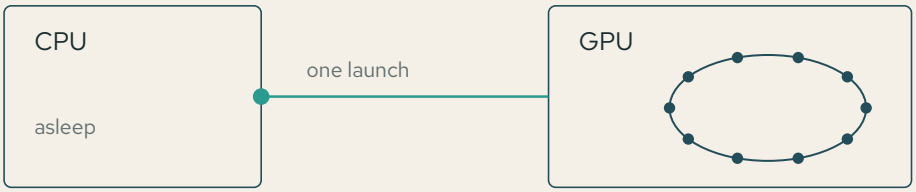
WHO DRIVES THE LOOP

ORDINARY TRAINING



The CPU sends the GPU every operation of every step, one by one.

QWENDEC



The CPU starts the loop and sleeps. Every step runs on the GPU.

Everything a step needs lives on the GPU: the data, their order, the learning-rate schedule, the optimiser and the logs. The CPU wakes once every 50 steps to save a checkpoint.

11 THE TRAINING LOOP

What was measured.

SHARE OF ONE CPU CORE

0.04%

During a full training run of 1,034 steps.

With the graphics card busy 99.7% of the time.

SIDE BY SIDE

A test of 16 steps	CPU core	GPU busy	Per step
Ordinary training	99.5%	56%	733 ms
The loop on the GPU	0.4%	96%	397 ms

WHAT HOLDS

- **The same result, bit for bit**
  The loop ends with exactly the weights ordinary training gives, step for step.
- **Stops and resumes exactly**
  A run continued in a fresh process ends in the identical state.
- **Used for every run in this report**
  1,034 steps for the first adapter, 810 for the single adapter, and each attempt in between.

TO KNOW

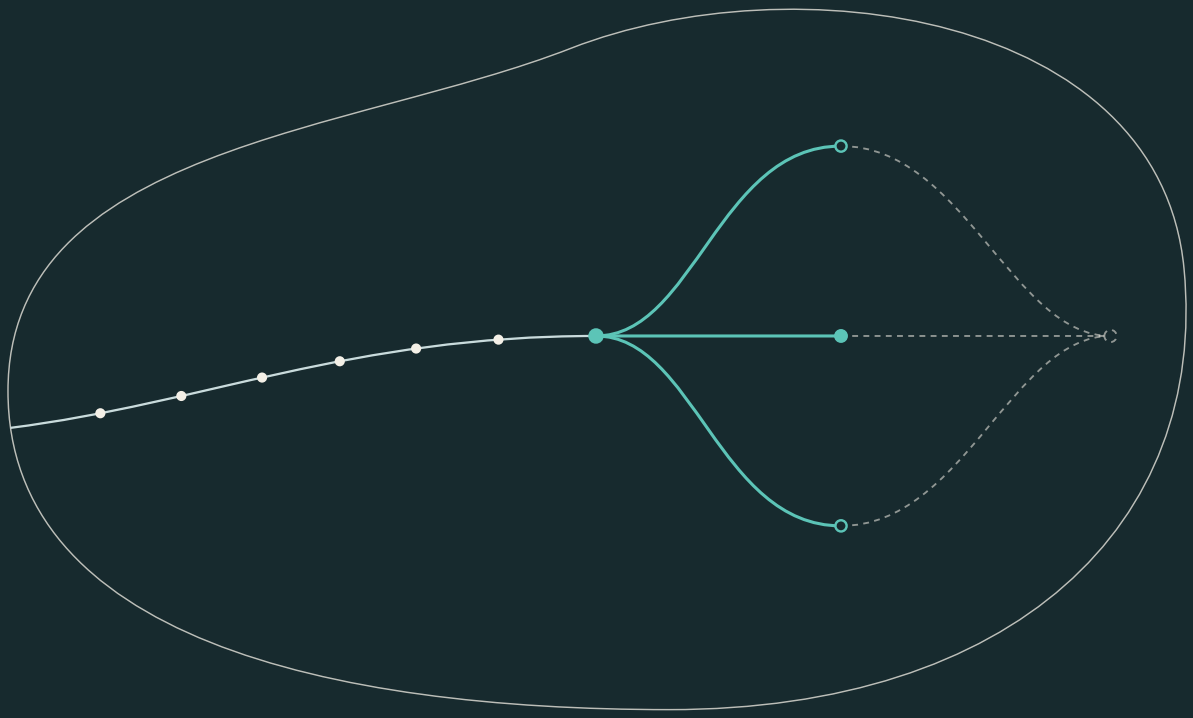
On the real training data the loop is not faster than ordinary training; the step is bound by the graphics card either way. What it removes is the CPU core held at 100%.

It relies on a feature of PyTorch that is still in beta and undocumented.

CHAPTER IV

What comes next.

Four pieces still to build, and two decisions waiting.



Still to build.

The decisions work when they are asked. The model does not ask yet.

Pending

- One adapter for both Noul and Choice

  Passed its rule on one run. Acceptance pending, page 18.

To build

- Reliable Score

  Score recognises when not to answer (70 of 80) and the ends of the scale (52 of 52). In-between values are poor: 11 of 67. The training cases hold only 19 distinct values. A test of a Score request as vectors is designed, not run.

- Model asks + uses answers

  Today each request is handed to the model. It has to learn to decide when to ask, to form the request and its pointers, and to use the result as it continues. This is the largest piece.

- Decision budget prediction

  A prediction, made at the start, of how much thinking a prompt deserves.

- Runtime + accounting

  One inference job that runs all of it and counts every position it computes.

In the repository these are called the joint adapter, Score, the native side, the allowance predictor and the runtime.

Two open decisions.

Both are the owner's to take. Nothing is running until they are.

1 *Accept the single adapter, or run it once more.*

It met both rules on one run, three parts by small margins. A second run with a different order of training steps would say whether that pass is stable.

Accept

The next stage builds on one adapter now. Its stability stays unknown.

Run once more

About 95 minutes of GPU time. The rule and its numbers stay as they are.

2 *Approve the Score request test.*

It asks whether a Score request can cross as vectors, with the rubric left where the model read it. The design and its pass rule are written. It needs about one hour of GPU time.

It is not expected to repair Score values. It would show whether the request is the problem. Three points in the design are also open: whether to run it before values are repaired, whether to test a pointer, and which adapter to start from.

No GPU time has been assigned to QwenDec since 12 November.

How to read the evidence.

How the work was run

- **Rule first**
  Each test's design and pass rule were published before its run.
- **One run each**
  The last step's weights are the only candidate. Nothing is retried after a failure. Three attempts are recorded as not passed.
- **Passing is not accepting**
  Applying the rule is mechanical. Accepting a result as the basis for the next stage is the owner's decision.
- **Open records**
  The code, the cases, the logs and the answer to every single question are public. No weights are.

What it does not show

A few fixed kinds of request. Six for Noul, two for Choice. The vectors only have to tell these apart. They do not yet carry a question's content.

Pointers are given. The model does not yet choose what to ask about.

Reliability across runs is open. Two runs of one design differed by 30 questions out of 1,182.

The cases are synthetic, built from templates and labelled by rule. A model reviewed the labels, not people.

General skill is unmeasured. Nothing here tests whether the adapter lost ability elsewhere.

Development figures only. A locked evaluation set exists and has never been read.

15 SOURCES

Sources.

Every figure in this report comes from the public repository.

github.com/veighnsche/QwenDec

BY CHAPTER

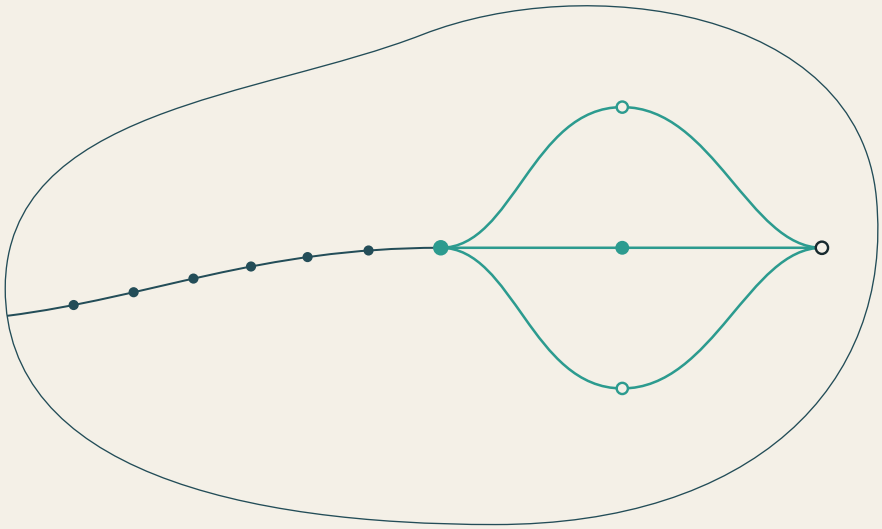
Overview and limits	`README.md`
What the system must do	`docs/requirements.md`
I. The model's own reading	`train/RESULTS.md, baseline/`
II. Noul, four attempts	`latent/RESULTS.md, RESULTS2.md, RESULTS3.md, RESULTS4.md`
II. Choice	`latent/CHOICE.md, RESULTS-CHOICE.md`
II. One adapter for both	`latent/JOINT.md, RESULTS-JOINT.md`
III. The training loop	`gpu_loop/RESULTS.md, train/RESULTS.md`
IV. Score request test	`latent/SCORE.md`

MODELS

The language model is Qwen3.5-4B. The decision adapter QwenDec started from is the published decision-4b. Neither is redistributed.

MEASURES

A question counts as right when its status is right and, where it has a value, the value is right. Score figures use the picked level, within 0.05 of the target.



COLOPHON

State of 12 November 2026. Development tests on synthetic cases.
Set in Newsreader, Red Hat Text and Red Hat Mono. Typeset with Typst.